

NetWatch SOC

Rapport Complet

Système de Détection Comportementale de Botnets

1 Présentation Générale du Projet

NetWatch SOC est une plateforme de détection de botnets orientée machine. L'unité d'analyse principale n'est pas le paquet réseau isolé, mais le profil complet d'une machine source pendant une fenêtre temporelle de cinq minutes. Cette approche permet de mieux détecter les comportements répétitifs, la dispersion réseau, les patterns périodiques et les anomalies DNS, qui sont typiques des activités botnet.

Le projet a été pensé comme une plateforme SOC compacte, interprétable et présentable dans un contexte PFE ou CV. Il combine un pipeline de traitement des logs, une extraction de features comportementales, des modèles de Machine Learning et de Deep Learning, un moteur de fusion hybride et un dashboard Django pour l'investigation.

2 Objectif du Projet

L'objectif principal du projet est de détecter automatiquement les machines normales, suspectes ou potentiellement infectées par un botnet, puis de fournir une explication compréhensible pour l'analyste.

Le projet doit produire pour chaque machine :

- une classe principale : normal, suspect, botnet
- un type d'activité : scanning, C2 communication, exfiltration, propagation, DDoS
- un score de risque de 0 à 100
- une explication textuelle

3 Problématique Traitée

Dans de nombreux environnements, les attaques botnet ne sont pas toujours visibles à travers une seule connexion réseau. Elles se manifestent souvent par :

- des connexions périodiques de type beaconing
- des accès DNS anormaux
- un grand nombre de destinations ou de ports
- des volumes de transfert sortants inhabituels
- des patterns de propagation latérale

La difficulté consiste donc à analyser le comportement global d'une machine, et non seulement un paquet ou un log isolé.

4 Architecture du Projet

Le projet est volontairement structuré avec peu de dossiers afin de garder une organisation simple et humaine :

- data/ — jeux de test et scénarios de démonstration
- ml_model/ — extraction de features, modèles, fusion et export PDF
- dashboard/ — interface Django, vues, frontend, interactions utilisateur
- soc_platform/ — configuration Django

→ exports/ — rapports PDF et documents générés

5 Pipeline de Travail

Le pipeline du projet suit cette logique :

- 1. Collecte de données réseau
- 2. Transformation en lignes exploitables
- 3. Agrégation par machine et par fenêtre temporelle
- 4. Extraction des features comportementales
- 5. Calcul de scores par les différents composants de détection
- 6. Fusion des résultats
- 7. Génération des alertes et explications
- 8. Affichage dans le dashboard
- 9. Export des rapports

6 Features Utilisées

6.1 Trafic global

- nombre total de connexions
- durée moyenne des connexions
- total bytes envoyés
- total bytes reçus
- ratio upload/download
- nombre total de paquets

6.2 Comportement réseau

- nombre d'IP destinations différentes
- nombre de ports contactés
- fréquence des connexions
- connexions simultanées
- connexions échouées

6.3 DNS

- nombre de requêtes DNS
- taux de réponses NXDOMAIN
- diversité des domaines
- fréquence des requêtes DNS

6.4 Temporel

- intervalle moyen entre connexions
- périodicité

- activité nocturne
- bursts de trafic

7 Méthodes Utilisées

7.1 Règles comportementales

Une première couche calcule un score explicable à partir de règles simples mais robustes. Cette couche permet de détecter rapidement des situations évidentes comme un scanning important, un ratio upload/download anormal ou un beaconing périodique.

7.2 IsolationForest

IsolationForest est utilisé pour la détection d'anomalies. Cet algorithme est pertinent car il permet de détecter des profils atypiques même lorsque toutes les attaques ne sont pas étiquetées. Il est rapide, robuste au bruit et bien adapté à une détection réseau par lots rapides.

7.3 RandomForestClassifier

RandomForestClassifier est utilisé pour la classification fine du type d'activité. Il permet de prédire si le comportement ressemble plutôt à du scanning, du C2, de l'exfiltration, de la propagation ou du DDoS. Son choix est justifié par sa robustesse, sa rapidité et sa capacité à fonctionner sur des features hétérogènes.

7.4 Dense Autoencoder

Le bloc Deep Learning repose sur un autoencoder dense. Ce modèle apprend une représentation de profils proches du trafic normal puis mesure l'erreur de reconstruction sur les nouvelles observations. Une erreur élevée peut indiquer une déviation comportementale inhabituelle.

7.5 Fusion hybride

Le score final combine le score comportemental basé sur règles, le score issu de la détection d'anomalies et le score issu de l'analyse profonde par reconstruction. Cette fusion renforce la robustesse globale du projet.

8 Dashboard et Visualisation

Le dashboard est développé avec Django. Il sert d'interface SOC simplifiée pour :

- afficher une vue globale
- consulter les machines analysées
- visualiser les alertes
- consulter des rapports
- voir des indicateurs globaux
- suivre les notifications non lues

Le dashboard a été nettoyé pour afficher directement l'information utile sans exposer inutilement des termes trop techniques à l'utilisateur final.

9 Sources de Données et Modes de Test

9.1 Jeux de données internes

- trafic normal
- scanning
- C2 beaconing
- exfiltration
- propagation
- DDoS

9.2 Collecte live — modes d'entrée supportés

- HTTP API
- UDP collector
- Zeek JSON
- Suricata EVE
- tshark
- NetFlow / Firewall

10 Points Forts du Projet

- ✓ L'analyse se fait par machine et non par paquet isolé
- ✓ La logique est interprétable
- ✓ L'architecture est simple à présenter
- ✓ Le dashboard permet une investigation rapide
- ✓ Les explications sont lisibles
- ✓ L'approche hybride augmente la robustesse
- ✓ Il est possible de tester le projet sans vrai réseau
- ✓ Le projet est facilement valorisable dans un CV ou un PFE

11 Limites et Points Non Idéaux

- Les modèles sont encore calibrés sur des profils synthétiques
- L'autoencoder n'est pas entraîné sur un grand corpus réel
- Il n'y a pas encore d'historisation poussée en base pour toutes les alertes
- La partie temps réel reste simplifiée par rapport à une plateforme SOC de production
- L'export PDF reste un export programmé manuellement et non un moteur de reporting industriel
- Les intégrations Zeek, Suricata et tshark sont prêtes conceptuellement mais pas encore automatisées de bout en bout

12 Améliorations Futures Possibles

- Entraîner les modèles sur des données réelles
- Ajouter une base de données dédiée aux incidents
- Introduire des WebSockets pour le temps réel natif
- Ajouter un système de comptes utilisateurs et de workflow SOC
- Ajouter la prise en charge directe de Zeek et Suricata
- Améliorer le deep learning avec des modèles séquentiels
- Introduire une véritable historisation des alertes lues et non lues
- Ajouter des filtres plus avancés par plage de dates, protocole et criticité

13 Méthode de Travail sur le Projet

Le projet a été développé en suivant une logique itérative :

- 1. Définition du besoin SOC
- 2. Choix de l'unité d'analyse par machine
- 3. Construction du pipeline de features
- 4. Intégration des algorithmes de détection
- 5. Fusion des scores
- 6. Création du dashboard
- 7. Amélioration du design
- 8. Ajout des rapports PDF
- 9. Ajustement progressif selon les besoins fonctionnels

Cette méthode a permis de garder le projet cohérent tout en l'améliorant étape par étape.

14 Technologies et Outils Utilisés

Catégorie	Outil / Technologie
Langage principal	Python
Framework web	Django
Machine Learning	scikit-learn
Deep Learning	TensorFlow / Keras
Frontend	HTML · CSS · JavaScript
Base de données	SQLite

15 Comment Tester sur un Réseau Réel

Un test simple sur réseau réel peut se faire avec une architecture légère :

- une machine de capture réseau
- une machine qui héberge NetWatch SOC
- une machine ou une VM qui génère le trafic

Le principe est :

- 1. Capturer les événements réseau
- 2. Reformater les flux dans le schéma attendu
- 3. Injecter les données vers /api/live/ingest ou en UDP
- 4. Observer les scores, les alertes et les explications dans le dashboard

16 Conclusion

NetWatch SOC est un projet solide de détection comportementale de botnets. Son point fort principal est de combiner une logique interprétable avec des algorithmes de Machine Learning et de Deep Learning dans une architecture simple, visible et exploitable.

Le projet est déjà convaincant pour une démonstration, un PFE ou une valorisation CV. Avec un entraînement sur données réelles et une meilleure industrialisation des intégrations réseau, il peut évoluer vers une plateforme beaucoup plus professionnelle.